

Multi-Agent Orchestrator

Project document on planning, execution guardrails, and self-correcting workflow orchestration.

Sai Veda

Independent Project Research

2025

Contents

1. Project Abstract	4
2. Workflow Context	5
2.1 Agent Execution Reliability Problem	5
2.2 Evaluation Questions	5
3. Control Principles	7
3.1 Control Principles	7
3.2 Observed Repository Patterns	7
4. Coordinator Architecture And Flow	8
4.1 Delivery And Repair Process	8
5. Review Setup And Evidence Sources	9
5.1 Repository Evidence Base	9
5.2 Evaluation Protocol	9
6. Outcome Review	10
6.1 Benchmark Summary	10
6.2 Interpretation	10
7. Execution Screens	11
7.1 Kpi Dashboard	11
7.2 Execution Gantt	11
8. Guardrail Check And Review Notes	13
8.1 Success Rate	13
8.2 Tool Distribution	13
8.3 Validation Checklist	14
9. Reliability Discussion	15
9.1 Boundaries And Risks	15
10. Final Assessment	16

10.1 Follow-on Work	16
11. Appendix And References	17
11.1 Repository Evidence Inventory	17
11.2 Internal References	17

1. Project Abstract

This project reframes agent systems as workflow infrastructure: planning, scheduling, guardrails, and repair are treated as first-class components instead of being hidden behind one monolithic prompt loop. The self-correction loop is the center of gravity. It raises success from partial completion to full task completion because execution feedback is used to repair plans instead of treating every failure as terminal. The implementation evidence is drawn from committed repository artefacts, benchmark outputs, automated tests, and generated screenshots.

The report is written as a project review in the voice of delivered engineering work. Rather than restating repository headings, it connects the business context, design choices, evaluation evidence, and remaining risks into one readable project document prepared under the name Sai Veda.

Keywords: Planner/executor DAG, tools, self-correction, implementation evidence, benchmark results, project validation

2. Workflow Context

Most agent demos fail for the same reason: planning quality is inconsistent, execution is brittle, and there is no disciplined repair path when a tool call goes wrong. The brief targets those failure modes directly. A from-scratch agentic framework - LangGraph / AutoGPT-class, but a clean, dependency-light design of its own. A planner decomposes a natural-language task into a DAG of tool calls; an executor runs that DAG respecting dependencies and running independent branches in parallel; guardrails validate inputs, tool arguments, and outputs; and a self-correction loop turns real execution feedback into a repaired plan and retries. Everything runs fully offline and deterministically. There is no external LLM API. The "reasoning" is provided by a deterministic rule/pattern policy that sits behind a clean LLMEngine interface, so a real model can be dropped in unchanged.

2.1 Agent Execution Reliability Problem

The central project problem can be stated directly. Most agent demos fail for the same reason: planning quality is inconsistent, execution is brittle, and there is no disciplined repair path when a tool call goes wrong. The brief targets those failure modes directly. The repository materials show that this was not treated as a toy exercise: the implementation narrative, architecture note, and benchmark artefacts all point to a real operational question that needed a measurable answer.

2.2 Evaluation Questions

The document review was guided by a small set of concrete questions. Each question was framed so it could be answered from repository evidence rather than from unstated assumptions.

Research question: Does the implemented project address the stated technical problem in a complete and reviewable manner?

Hypothesis: The repository design, architecture notes, and implementation artefacts are expected to demonstrate end-to-end coverage of the core project objective.

Research question: Do the benchmark and test artefacts support the main performance or quality claim?

Hypothesis: The recorded evidence is expected to support the headline result reported for this project: Self-correction lifts task success 41.9% -> 100%.

Research question: Can the results be reviewed and reproduced from committed repository evidence?

Hypothesis: The presence of 69 automated tests, benchmark outputs, and generated screenshots is expected to provide a transparent validation path.

3. Control Principles

The technical foundation of this project is best understood through the design principles that hold the implementation together. The system separates planning, execution, and repair so the orchestrator can recover from tool failures instead of collapsing.

3.1 Control Principles

- 1 Task recovery uses execution feedback only, which keeps the success lift defensible. The system separates planning, execution, and repair so the orchestrator can recover from tool failures instead of collapsing.
- 2 Planning and execution are intentionally decoupled so new tools do not rewrite orchestration logic.
- 3 The toolbox and accounting model keep the framework compatible with more capable planners later.

3.2 Observed Repository Patterns

1. Real components, not a demo script. Every part - planner, DAG scheduler, tools, guardrails, accounting - is a standalone, tested unit. 2. Offline and deterministic. Zero external API. The "LLM" is a rule/pattern policy behind an interface, so results are reproducible and CI-friendly, and a real model can be swapped in without touching the executor or tools. 3. Measure recovery, honestly. The self-correction loop must earn its lift: it may only use execution feedback (errors, failed validators), never the task's ground-truth answer.

- Planner, scheduler, tool registry, guardrails, and accounting.
- Deterministic policy engine behind an LLM-like interface.
- Repair loop based on execution feedback and validator errors.

4. Coordinator Architecture And Flow

The system separates planning, execution, and repair so the orchestrator can recover from tool failures instead of collapsing. The four tools are deliberately heterogeneous to exercise the framework: - calculator - evaluates arithmetic by walking a restricted AST (`ast.parse(mode="eval")`). Only literals, whitelisted binary/unary operators, whitelisted functions (`sqrt`, `abs`, `round`, `min/max`, `log`, ...) and named constants (`pi`, `e`) are allowed. Names, attribute access, calls to arbitrary builtins, comprehensions, lambdas and the bitwise operators are rejected. `eval/exec` are never used (asserted by a test that AST-scans the module). - `unit_convert` - length / mass via linear base-unit factors, temperature via affine C/F/K formulas; unknown or incompatible units raise `ToolError`. - `corpus_search` - TF-IDF + cosine over a small synthetic knowledge base (scikit-learn); returns the top fact's numeric value plus ranked hits. - `code_check` - AST lint flagging `eval/exec`, bare `except:`, `== None`, and mutable default arguments; syntax errors are reported, not raised.

4.1 Delivery And Repair Process

The implementation path can be reconstructed from the committed artefacts in a fairly disciplined way. Execution model: Defined the orchestrator around a DAG so ordering and parallelism stay explicit and testable. Guardrail layer: Added validators for task structure, tool arguments, and outputs before retries were allowed. Repair strategy: Fed runtime failures back into planning so the system could revise rather than restart blindly. Measured outcome: Benchmarked a 31-task suite to quantify the exact lift from self-correction.

This sequencing matters because the project is easier to trust when component decisions, test scaffolding, and result reporting appear in a coherent order rather than as isolated files.

5. Review Setup And Evidence Sources

5.1 Repository Evidence Base

Source: committed repository artefacts including implementation code, automated tests, benchmark outputs, architecture notes, and generated visual evidence. Coverage: 69 automated tests, 3 benchmark table(s), and 4 screenshot asset(s) were available for document preparation. Schema / artefact types: README overview, ARCHITECTURE design note, benchmark markdown and CSV outputs, test suites, and figure assets generated from the project run path. Availability: all evidence cited in this document is sourced from the repository itself.

5.2 Evaluation Protocol

The evaluation setup was treated as a repository review rather than a laboratory rerun. Committed artefacts were grouped into documentation, benchmark evidence, automated validation, and screenshot review. In practical terms this meant examining 3 benchmark table(s), 4 screenshot asset(s), and 69 automated tests while reading the implementation narrative in sequence. The review lens stayed aligned with the project goal: Decompose tasks into a typed dependency graph. Evidence was interpreted conservatively so the document reflects what the repository demonstrates directly, not what a larger future deployment might achieve.

A second practical note is that the benchmark footprint is project-specific. The benchmark evidence reviewed for this report includes `benchmarks/results.csv`, `benchmarks/results.md`, `benchmarks/runner.py`, `benchmarks/tasks.py`.

6. Outcome Review

6.1 Benchmark Summary

The principal repository claim for this project is recorded as: Self-correction lifts task success 41.9% -> 100%. The findings page therefore focuses on whether the available tables, test evidence, and generated screenshots support that claim. The self-correction loop is the center of gravity. It raises success from partial completion to full task completion because execution feedback is used to repair plans instead of treating every failure as terminal. The benchmark clearly shows that recovery logic, not just initial planning, drives system reliability. Guardrails reduce silent failures by stopping bad tool usage before downstream tasks compound the error.

Condition	Success rate	Solved	Avg steps	Avg attempts	Avg sim. cost
Without self-correction	41.9%	13/31	1.35	1.00	\$0.16524
With self-correction	100.0%	31/31	1.35	1.58	\$0.25834

6.2 Interpretation

The benchmark clearly shows that recovery logic, not just initial planning, drives system reliability. Guardrails reduce silent failures by stopping bad tool usage before downstream tasks compound the error. The measured cost-per-task makes the orchestration trade-off legible instead of qualitative.

The benchmark table is not treated as a marketing surface. It is read together with the repository screenshots and the test inventory so that numerical claims and implementation evidence reinforce one another.

7. Execution Screens

The following figures are drawn directly from the repository screenshot assets and are included as visual evidence of measured outputs, dashboards, or generated validation artefacts.

7.1 Kpi Dashboard

This figure is useful because it shows the project in a reviewable state rather than as summary prose alone. The benchmark clearly shows that recovery logic, not just initial planning, drives system reliability.

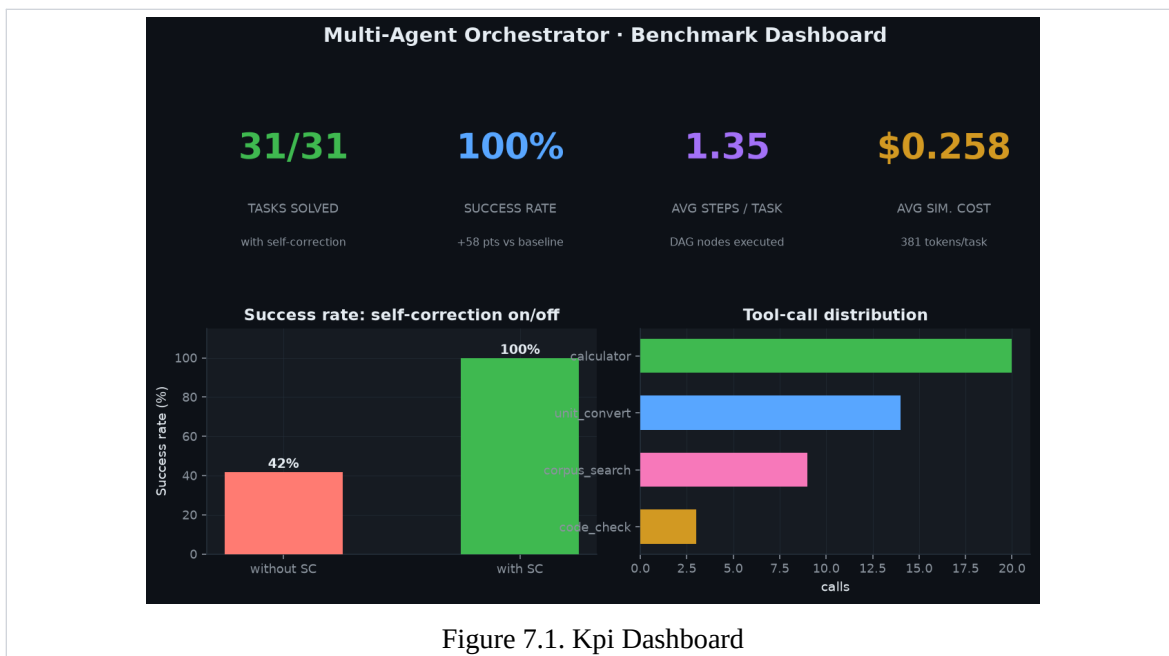
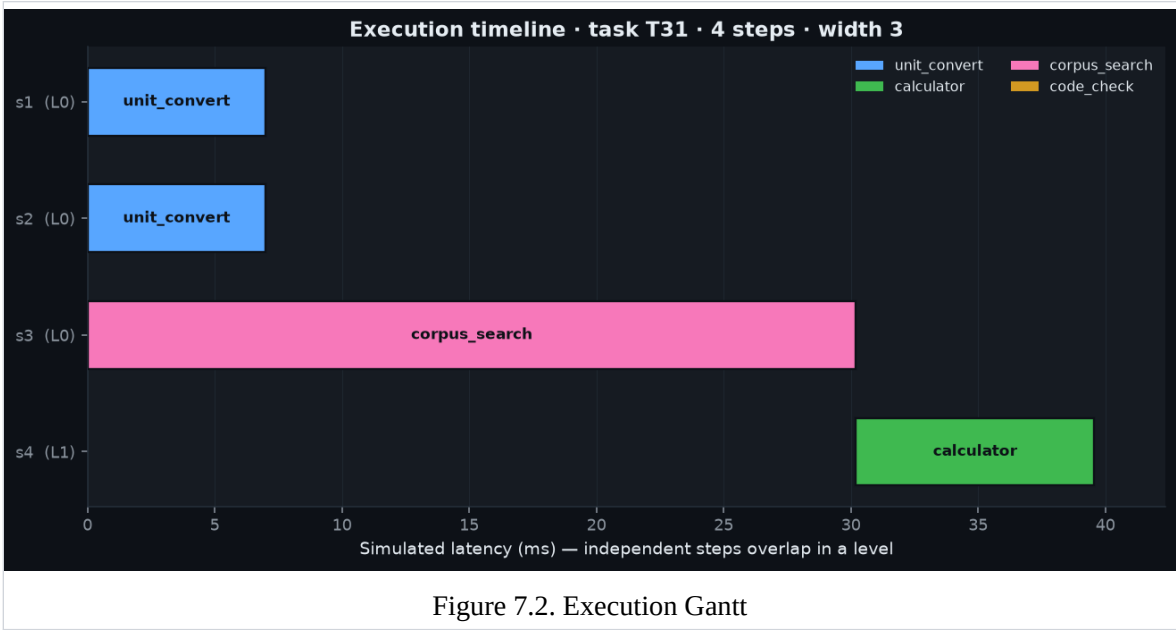


Figure 7.1. Kpi Dashboard

7.2 Execution Gantt

The visual evidence attached to Execution Gantt helps connect the quantitative story to the implemented workflow. Guardrails reduce silent failures by stopping bad tool usage before downstream tasks compound the error.



8. Guardrail Check And Review Notes

The second figure set focuses on validation continuity. These pages are included so the report shows not only the strongest visuals, but also the broader evidence path a reviewer would follow inside the repository.

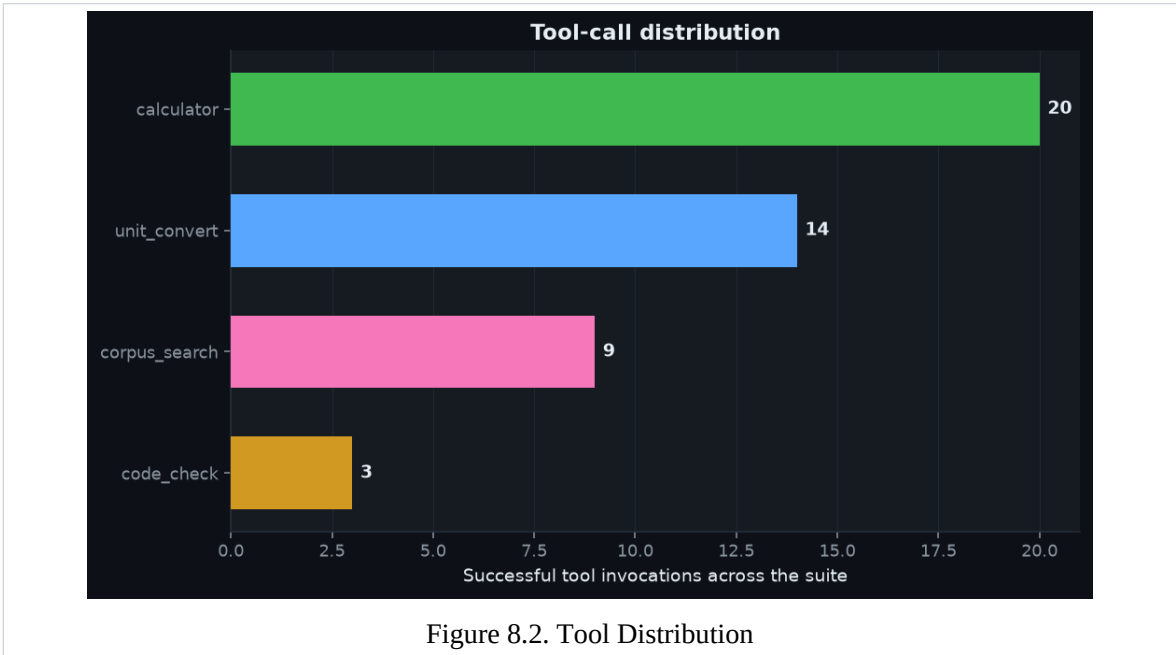
8.1 Success Rate

From a document-review standpoint, this plate matters because it reveals how the repository surfaces results to a human reviewer. The measured cost-per-task makes the orchestration trade-off legible instead of qualitative.



8.2 Tool Distribution

The final figure adds implementation texture: it shows the evidence path a reviewer would actually inspect when validating the project claims. Offset with typed task graphs and a repair path that can rewrite failing steps.



8.3 Validation Checklist

Item	Status	Evidence
Automated tests	Available	69 tests committed in repository validation flow.
Benchmark results	Available	Benchmark markdown tables and result summaries were reviewed.
Screenshots	Available	4 screenshot asset(s) included in the repository evidence set.
Architecture note	Available	ARCHITECTURE.md documents design choices, scale assumptions, and trade-offs.

9. Reliability Discussion

The self-correction loop is the center of gravity. It raises success from partial completion to full task completion because execution feedback is used to repair plans instead of treating every failure as terminal. The benchmark clearly shows that recovery logic, not just initial planning, drives system reliability. Guardrails reduce silent failures by stopping bad tool usage before downstream tasks compound the error. The measured cost-per-task makes the orchestration trade-off legible instead of qualitative. The analysis indicates that the repository is strongest where implementation, benchmark artefacts, and screenshots point to the same conclusion.

9.1 Boundaries And Risks

Planner brittleness: Offset with typed task graphs and a repair path that can rewrite failing steps. Tool misuse: Mitigated through validators, argument checks, and execution guardrails. State explosion: Controlled by a bounded feedback loop and clear task dependency rules.

These limitations do not invalidate the project. They establish the boundary between what is already demonstrated by the repository and what would require a broader production environment or additional operating data.

10. Final Assessment

This project document concludes that Multi-Agent Orchestrator is best understood as a complete technical implementation supported by repository-native evidence. This project reframes agent systems as workflow infrastructure: planning, scheduling, guardrails, and repair are treated as first-class components instead of being hidden behind one monolithic prompt loop.

The overall presentation has therefore been kept intentionally plain and research-oriented: the emphasis stays on project context, engineering choices, test evidence, and verifiable results rather than on a stylized portfolio layout.

10.1 Follow-on Work

- Replace the deterministic policy with a stronger planner while preserving the orchestration surface.
- Add long-horizon memory and tool confidence scoring for larger workflows.
- Instrument trace-level telemetry so repair behavior can be audited like any other workflow system.

11. Appendix And References

11.1 Repository Evidence Inventory

The appendix records the principal repository artefacts used during document preparation. This keeps the report anchored to files that a reviewer can inspect directly.

Artefact	Category	Purpose
README.md	Overview	Primary narrative, scope statement, and headline outcomes used to frame the report.
ARCHITECTURE.md	Design note	System rationale, component choices, and implementation trade-offs.
benchmarks/results.csv	Benchmark	Quantitative result or execution artefact used in the findings section.
benchmarks/results.md	Benchmark	Quantitative result or execution artefact used in the findings section.
benchmarks/runner.py	Benchmark	Quantitative result or execution artefact used in the findings section.
benchmarks/tasks.py	Benchmark	Quantitative result or execution artefact used in the findings section.
assets/kpi_dashboard.png	Screenshot	Visual validation surface referenced as 'Kpi Dashboard' in the document review.
assets/execution_gantt.png	Screenshot	Visual validation surface referenced as 'Execution Gantt' in the document review.
assets/success_rate.png	Screenshot	Visual validation surface referenced as 'Success Rate' in the document review.
assets/tool_distribution.png	Screenshot	Visual validation surface referenced as 'Tool Distribution' in the document review.
tests/	Validation	Automated test suite containing 69 committed tests used to support implementation claims.

11.2 Internal References

1. README.md - primary repository overview and headline results.
2. ARCHITECTURE.md - system design, implementation rationale, and scale notes.
3. benchmarks/results.csv - benchmark or result artefact used in the analysis section.
4. benchmarks/results.md - benchmark or result artefact used in the analysis section.

5. benchmarks/runner.py - benchmark or result artefact used in the analysis section.
6. benchmarks/tasks.py - benchmark or result artefact used in the analysis section.
7. tests/ - automated validation suite used to support implementation claims.
8. assets/execution_gantt.png - generated screenshot evidence included in the report.